

Project 5: Enterprise Document Intelligence System

Project Report

A RAG-based enterprise document intelligence application for policies, contracts, and reports.

1. Problem

Organizations may have thousands of PDFs containing policies, contracts, reports, and other operational knowledge. Traditional file search can locate documents but does not reliably answer natural-language questions across a large corpus, nor does it automatically provide the relevant evidence. This project addresses that problem with retrieval-augmented generation (RAG).

2. Architecture

The application uses a Python-based pipeline. A user uploads PDFs through Streamlit. Each PDF is extracted page by page, split into overlapping chunks, embedded with an OpenAI embedding model, and persisted in ChromaDB. For a question, semantic similarity retrieval selects the most relevant chunks. The retrieved context is passed to an OpenAI chat model with a prompt that requires grounded answers and source references.

Layer	Component	Responsibility
Interface	Streamlit	Upload, indexing, questions, summaries, analysis
Ingestion	pypdf + splitter	Extract PDF text and create overlapping chunks
Embeddings	OpenAI Embeddings	Convert chunks/questions into semantic vectors
Storage	ChromaDB	Persist and retrieve relevant document chunks
Generation	OpenAI Chat Model	Generate grounded answers and summaries

3. Prompt Design

RAG prompt: The model is instructed to answer only from retrieved context, explicitly say when the evidence does not support an answer, and cite retrieved passages using [Source N]. This reduces unsupported answers and makes the response traceable to the supplied documents.

Document analyst prompt: The analyst separates explicit facts from interpretation, flags ambiguity, and identifies missing information. This is useful for contracts and policy reviews where a user needs structured analysis rather than a simple summary.

Executive-summary prompt: The summary workflow emphasizes purpose, key points, risks or obligations, important dates/numbers, and follow-up questions. The input is bounded to a practical context window in the baseline implementation.

4. Evaluation

Evaluation should use a representative question set covering policies, contracts, and reports. Key measures are retrieval relevance, answer faithfulness, citation correctness, latency, and summarization coverage. Include unanswerable questions to test whether the system avoids inventing information. A production evaluation can use a labeled dataset of question, expected evidence, and acceptable answer criteria.

Suggested evaluation matrix

Metric	What to check
Retrieval relevance	Are the top retrieved chunks actually relevant to the question?
Faithfulness	Does the answer stay supported by retrieved text?
Citation correctness	Do source references point to the passages used?
Latency	Time from question submission to displayed response
Summary coverage	Are important facts, obligations, risks, and dates preserved?

5. Project Design and User Flow

The application is intentionally modular. `document_processor.py` handles PDF extraction and chunking; `rag_pipeline.py` handles embeddings, vector storage, retrieval, and question answering; `summary.py` provides summarization; `analyst.py` provides document analysis; and `app.py` exposes these functions through Streamlit.

Typical flow: (1) select Policies, Contracts, or Reports; (2) upload PDFs; (3) index them; (4) ask a natural-language question; (5) inspect the answer and retrieved passages; or (6) paste a document excerpt into the analyst area for an executive summary or deeper analysis.

6. Challenges

PDF extraction is a major practical challenge because scanned PDFs may contain no machine-readable text and complex layouts can degrade extraction quality. Retrieval quality also depends on chunk size, overlap, embedding quality, and corpus metadata. A generative model may produce incomplete or unsupported answers when retrieval is weak, so grounding instructions and evaluation are important.

Enterprise deployments introduce additional requirements: authentication, role-based access, document-level permissions, encryption, audit logs, retention policies, monitoring, and careful handling of confidential data. The baseline project intentionally keeps these concerns outside the minimal demo implementation.

7. Future Enhancements

- OCR for scanned PDFs and image-heavy documents
- SSO and role-based/document-level access control
- Hybrid BM25 + vector retrieval and reranking
- Document versioning and effective-date filtering
- PII detection and redaction
- Human feedback and answer correction workflows
- Batch generation of executive summaries
- Automated RAG evaluation and observability

8. Deliverables

The accompanying project folder contains the Python source code, requirements, environment template, README, Mermaid architecture diagram, 3–5 minute demo script, and this PDF report.

Technology stack: Python • OpenAI API • LangChain • ChromaDB • Streamlit